

GRAPHS

Graphs and Graph Models:

Graphs: A graph $G = (V, E)$ consists of V , a nonempty set of vertices (or nodes) and E , a set of edges. Each edge has either one or two vertices associated with it, called its endpoints. An edge is said to connect its endpoints.

Loop: In a graph an edge has same vertex as its end vertices is called loop.

Parallel edges: In a graph, allows more than one edge (2 or more) associated with a pair of vertices are called parallel edges.

Order and size of a graph: The number of vertices is called order of graph. The number of edges is called size of the graph.

Undirected graphs: The edges of graphs are undirected then that graphs are called undirected graphs.

Directed Graphs:

A directed graph (or digraph) (V, E) consists of a nonempty set of vertices V and a set of directed edges (or arcs) E . Each directed edge is associated with an ordered pair of vertices. The directed edge associated with the ordered pair (u, v) is said to start at u and end at v .

Simple directed graphs:

A directed graph has no loops and has no multiple directed edges, it is called a simple directed graph. Because a simple directed graph has at most one edge associated to each ordered pair of vertices (u, v) , we call (u, v) an edge if there is an edge associated to it in the graph.

Directed multigraphs:

Directed graphs that may have multiple directed edges from a vertex to a second (possibly the same) vertex such graphs are called directed multigraphs.

Graph Models:

Graphs are used in a wide variety of models. We will discuss this section by describing some diverse graph models for some interesting applications.

Social Networks:

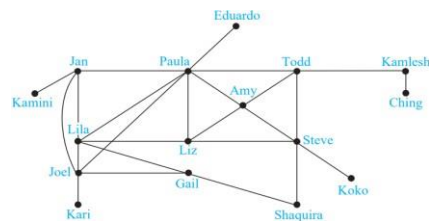
Graphs are extensively used to model social structures based on different kinds of relationships between people or groups of people. These social structures, and the graphs that represent them, are known as social networks. In these graph models, individuals or organizations are represented by vertices; relationships between individuals or organizations are represented by edges. The study of social networks is an extremely active multidisciplinary area, and many different types of relationships between people have been studied using them.

Example 1:

Acquaintanceship and Friendship Graphs : We can use a simple graph to represent whether two people know each other, that is, whether they are acquainted, or whether they are friends (either in the real world in the virtual world via a social networking site such as Facebook).

Each person in a particular group of people is represented by a vertex. An undirected edge is used to connect two people when these people know each other, when we are concerned only with acquaintanceship, or whether they are friends. No multiple edges and usually no loops are used. (If we want to include the notion of self-knowledge, we would include loops.)

A small acquaintanceship graph is shown in Figure .

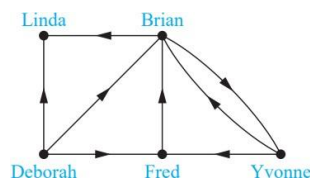


Example 2 :

Influence Graphs: In studies of group behavior it is observed that certain people can influence the thinking of others. A directed graph called an influence graph can be used to model this behavior.

Each person of the group is represented by a vertex. There is a directed edge from vertex a to vertex b when the person represented by vertex a can influence the person represented by vertex b . This graph does not contain loops and it does not contain multiple directed edges.

An example of an influence graph for members of a group is shown in Figure.



In the group modeled by this influence graph, Deborah cannot be influenced, but she can influence Brian, Fred, and Linda. Also, Yvonne and Brian can influence each other.

Example 3:

Collaboration Graphs: A collaboration graph is used to model social networks where two people are related by working together in a particular way. Collaboration graphs are simple graphs, as edges in these graphs are undirected and there are no multiple edges or loops.

Vertices in these graphs represent people; two people are connected by an undirected edge when the people have collaborated. There are no loops nor multiple edges in these graphs.

The Hollywood graph is a collaborator graph that represents actors by vertices and connects two actors with an edge if they have worked together on a movie or television show.

In an academic collaboration graph, vertices represent people (perhaps restricted to members of a certain academic community), and edges link two people if they have jointly published a paper.

Communication Networks:

We can model different communications networks using vertices to represent devices and edges to represent the particular type of communications links of interest.

Example 4:

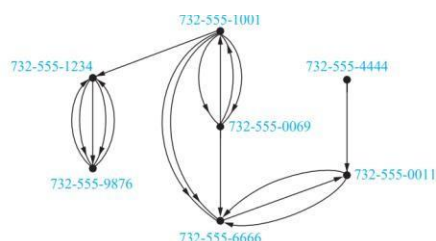
Call Graphs : Graphs can be used to model telephone calls made in a network, such as a long distance telephone network.

In particular, a directed multigraph can be used to model calls where each telephone number is represented by a vertex and each telephone call is represented by a directed edge.

The edge representing a call starts at the telephone number from which the call was made

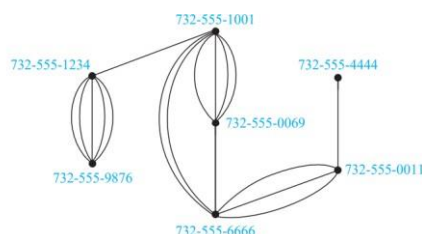
and ends at the telephone number to which the call was made. We need directed edges because the direction in which the call is made matters. We need multiple directed edges because we want to represent each call made from a particular telephone number to a second number.

A small telephone call graph is displayed in Figure



This graph representing call graph of seven telephone numbers. This graph shows, for instance, that three calls have been made from 732-555-1234 to 732-555-9876 and two in the other direction, but no calls have been made from 732-555-4444 to any of the other six numbers except 732-555-0011.

When we care only whether there has been a call connecting two telephone numbers, we use an undirected graph with an edge connecting telephone numbers when there has been a call between these numbers. This version of the call graph is displayed in Figure



Information Networks:

Graphs can be used to model various networks that link particular types of information. Here, we will describe how to model the World Wide Web using a graph. We will also describe how to use a graph to model the citations in different types of documents.

Example 5:

The Web Graph: The World Wide Web can be modeled as a directed graph where each Web page is represented by a vertex and where an edge starts at the Web page a and ends at the Web page b if there is a link on a pointing to b . Because new Web pages are created and others removed somewhere on the Web almost every second, the Web graph changes on an almost continual basis.

Example 6:

Citation Graphs: Graphs can be used to represent citations in different types of documents, including academic papers, patents, and legal opinions.

In such graphs, each document is represented by a vertex, and there is an edge from one document to a second document if the first document cites the second in its citation list. (In an academic paper, the citation list is the bibliography, or list of references; in a patent it is the list of previous patents that are cited; and in a legal opinion it is the list of previous opinions cited.)

A citation graph is a directed graph without loops or multiple edges.

Software Design Applications:

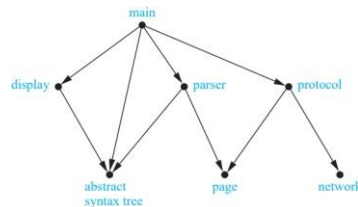
Graph models are useful tools in the design of software. We will briefly describe two of these models here.

Example 7:

Module Dependency Graphs: One of the most important tasks in designing software is how to structure a program into different parts, or modules. Understanding how the different modules of a program interact is essential not only for program design, but also for testing and maintenance of the resulting software. A module dependency graph provides a useful tool for understanding how different modules of a program interact.

In a program dependency graph, each module is represented by a vertex. There is a directed edge from a module to a second module if the second module depends on the first.

An example of a program dependency graph for a web browser is shown in Figure.

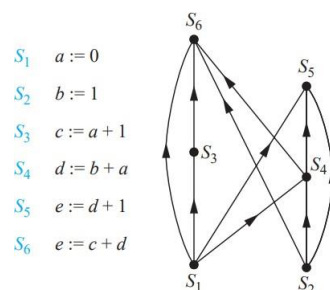


Example 8:

Precedence Graphs and Concurrent Processing: Computer programs can be executed more rapidly by executing certain statements concurrently. It is important not to execute a statement that requires results of statements not yet executed. The dependence of statements on previous statements can be represented by a directed graph.

Each statement is represented by a vertex, and there is an edge from one statement to a second statement if the second statement cannot be executed before the first statement. This resulting graph is called a precedence graph.

A computer program and its graph are displayed in Figure.



For instance, the graph shows that statement S_5 cannot be executed before statements S_1 , S_2 , and S_4 are executed.

Transportation Networks:

We can use graphs to model many different types of transportation networks, including road, air, and rail networks, as well shipping networks.

Example 9:

Airline Routes : We can model airline networks by representing each airport by a vertex. In particular, we can model all the flights by a particular airline each day using a directed edge to represent each flight, going from the vertex representing the departure airport to the vertex representing the destination airport. The resulting graph will generally be a directed multigraph, as there may be multiple flights from one airport to some other airport during the same day.

Example 10 :

Road Networks: Graphs can be used to model road networks. In such models, vertices represent intersections and edges represent roads.

When all roads are two-way and there is at most one road connecting two intersections,

we can use a simple undirected graph to model the road network.

However, we will often want to model road networks when some roads are one-way and when there may be more than one road between two intersections. To build such models, we use undirected edges to represent two-way roads and we use directed edges to represent one-way roads.

Multiple undirected edges represent multiple two-way roads connecting the same two intersections. Multiple directed edges represent multiple one-way roads that start at one intersection and end at a second intersection. Loops represent loop roads. Mixed graphs are needed to model road networks that include both one-way and two-way roads.

Biological Networks:

Many aspects of the biological sciences can be modeled using graphs.

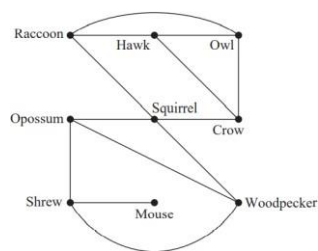
Example 11:

Niche Overlap Graphs: This Ecology Graphs are used in many models involving the interaction of different species of animals. For instance, the competition between species in an ecosystem can be modeled using a niche overlap graph.

Each species is represented by a vertex. An undirected edge connects two vertices if the two species represented by these vertices compete (that is, some of the food resources they use are the same).

A niche overlap graph is a simple graph because no loops or multiple edges are needed in this model.

The graph in Figure models the ecosystem of a forest.



We see from this graph that squirrels and raccoons compete but that crows and shrews do not.

Example 12:

Protein Interaction Graphs: A protein interaction in a living cell occurs when two or more proteins in that cell bind to perform a biological function. Because protein interactions are crucial for most biological functions, many scientists work on discovering new proteins and understanding interactions between proteins.

Protein interactions within a cell can be modeled using a protein interaction graph (also called a protein–protein interaction network), an undirected graph in which each protein is represented by a vertex, with an edge connecting the vertices representing each pair of proteins that interact.

It is a challenging problem to determine genuine protein interactions in a cell, as experiments often produce false positives, which conclude that two proteins interact when they really do not. Protein interaction graphs can be used to deduce important biological information, such as by identifying the most important proteins for various functions and the functionality of newly discovered proteins. Because there are thousands of different proteins in a typical cell, the protein interaction graph of a cell is extremely large and complex.

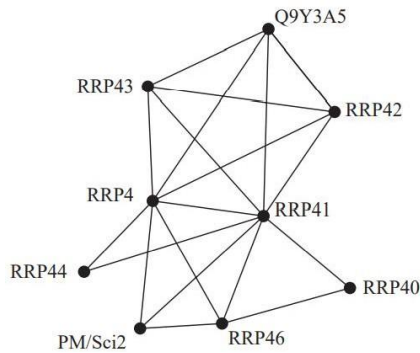


Figure illustrates a module of the protein interaction graph described in [Bo04], comprising the complex of proteins that degrade RNA in human cells.

Tournaments:

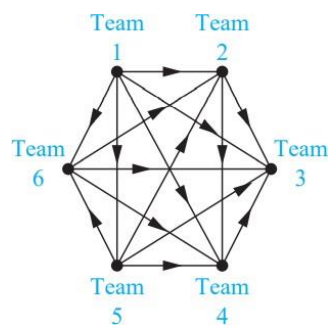
We now give some examples that show how graphs can also be used to model different kinds of tournaments.

Example 13:

Round-Robin Tournaments: A tournament where each team plays every other team exactly once and no ties are allowed is called a round-robin tournament. Such tournaments can be modeled using directed graphs where each team is represented by a vertex. Note that (a, b) is an edge if team a beats team b .

This graph is a simple directed graph, containing no loops or multiple directed edges (because no two teams play each other more than once).

Such a directed graph model is presented in Figure.

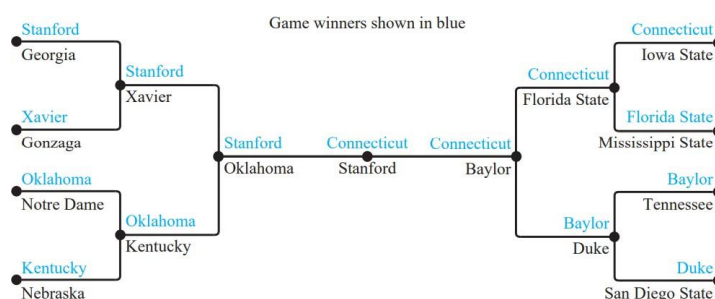


We see that Team 1 is undefeated in this tournament, and Team 3 is winless.

Example 14:

Single-Elimination Tournaments: A tournament where each contestant is eliminated after one loss is called a single-elimination tournament.

We can model such a tournament using a vertex to represent each game and a directed edge to connect a game to the next game the winner of this game played in.



The graph in Figure represents the games played by the final 16 teams in the 2010 NCAA women's basketball tournament.

Graph Terminology and Special Types of Graphs:

Basic Terminology

First, we give some terminology that describes the vertices and edges of undirected graphs.

Incident : When a vertex “v” is an end vertex of some edge “e”, “v” and “e” are said to be incident with each other.

Adjacent vertices: In a graph G , two vertices “a” and “b” are said to be adjacent (or neighbors) if they are end vertices of the same edge.

Neighborhood of vertex:

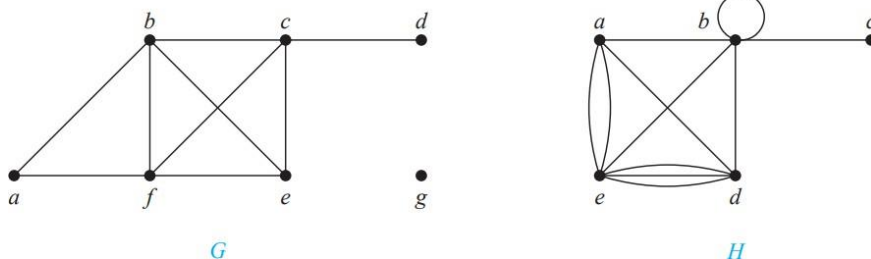
The set of all neighbors of a vertex v of $G = (V, E)$, denoted by $N(v)$, is called the neighborhood of v .

If A is a subset of V , we denote by $N(A)$ the set of all vertices in G that are adjacent to at least one vertex in A . So, $N(A) = \bigcup_{v \in A} N(v)$.

Degree of a vertex:

The degree of a vertex in an undirected graph is the number of edges incident with it, except that a loop at a vertex contributes twice to the degree of that vertex. The degree of the vertex v is denoted by $\deg(v)$.

Example 1: What are the degrees and what are the neighborhoods of the vertices in the graphs G and H displayed in Figure?



Solution: In G the degree of vertices are,

$\deg(a) = 2$, $\deg(b) = \deg(c) = \deg(f) = 4$, $\deg(d) = 1$, $\deg(e) = 3$, and $\deg(g) = 0$.

The neighborhoods of these vertices are $N(a) = \{b, f\}$, $N(b) = \{a, c, e, f\}$, $N(c) = \{b, d, e, f\}$, $N(d) = \{c\}$, $N(e) = \{b, c, f\}$, $N(f) = \{a, b, c, e\}$, and $N(g) = \emptyset$.

In H the degree of vertices are,

$\deg(a) = 4$, $\deg(b) = \deg(e) = 6$, $\deg(c) = 1$, and $\deg(d) = 5$.

The neighborhoods of these vertices are,

$N(a) = \{b, d, e\}$, $N(b) = \{a, b, c, d, e\}$, $N(c) = \{b\}$, $N(d) = \{a, b, e\}$, and $N(e) = \{a, b, d\}$.

Isolated and pendant of vertex:

A vertex of degree zero is called isolated. It follows that an isolated vertex is not adjacent to any vertex. Vertex g in graph G in Example 1 is isolated.

A vertex is pendant if and only if it has degree one. Consequently, a pendant vertex is adjacent to exactly one other vertex. Vertex d in graph G in Example 1 is pendant.

Theorem 1: The Handshaking Theorem:

Let $G = (V, E)$ be an undirected graph with m edges.

Then $2m = \sum_{v \in V} \deg(v)$.

(Note that this applies even if multiple edges and loops are present).

Example 3: How many edges are there in a graph with 10 vertices each of degree six?

Solution: The number of vertices = 10 and each vertex of degree = 6

Then sum of the degrees of the vertices is $6 \cdot 10 = 60$,

it follows that $2m = 60$

where m is the number of edges.

Therefore, $m = 60/2$

$= 30$.

Theorem 2: An undirected graph has an even number of vertices of odd degree.

Proof: Let V_1 and V_2 be the set of vertices of even degree and the set of vertices of odd degree, respectively, in an undirected graph $G = (V, E)$ with m edges. Then

$$2m = \sum_{v \in V} \deg(v) = \sum_{v \in V_1} \deg(v) + \sum_{v \in V_2} \deg(v).$$

Because $\deg(v)$ is even for $v \in V_1$, the first term in the right-hand side of the last equality is even. Furthermore, the sum of the two terms on the right-hand side of the last equality is even, because this sum is $2m$. Hence, the second term in the sum is also even. Because all the terms in this sum are odd, there must be an even number of such terms. Thus, there is an even number of vertices of odd degree.

Adjacent vertex in directed graph:

When (u, v) is an edge of the graph G with directed edges, u is said to be adjacent to v and v is said to be adjacent from u .

Initial and terminal vertex in directed graph:

The vertex u is called the initial vertex of (u, v) , and v is called the terminal or end vertex of (u, v) . The initial vertex and terminal vertex of a loop are the same.

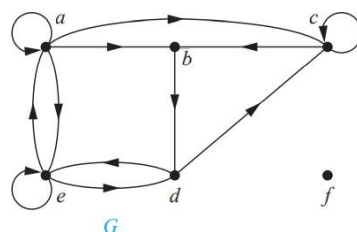
In-degree and out-degree of vertex:

In a graph with directed edges the in-degree of a vertex v , denoted by $\deg^-(v)$, is the number of edges with v as their terminal vertex.

In a graph with directed edges the out-degree of a vertex v , denoted by $\deg^+(v)$, is the number of edges with v as their initial vertex.

Note: A loop at a vertex contributes 1 to both the in-degree and the out-degree of this vertex.

Example: Find the in-degree and out-degree of each vertex in the graph G with directed edges shown in Figure.



Solution:

The in-degrees in G are $\deg^-(a) = 2$, $\deg^-(b) = 2$, $\deg^-(c) = 3$, $\deg^-(d) = 2$, $\deg^-(e) = 3$, and $\deg^-(f) = 0$.

The out-degrees are $\deg^+(a) = 4$, $\deg^+(b) = 1$, $\deg^+(c) = 2$, $\deg^+(d) = 2$, $\deg^+(e) = 3$, and $\deg^+(f) = 0$.

Theorem 3: Let $G = (V, E)$ be a graph with directed edges. The

$$\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) = |E|.$$

Types of Graphs

Infinite graph: The set of vertices V of a graph G may be infinite. A graph with an infinite vertex set or an infinite number of edges is called an infinite graph.

Finite graphs: A graph with a finite vertex set and a finite edge set is called a finite graph.

Simple graphs: A graph in which each edge connects two different vertices and where no two edges connect the same pair of vertices is called a simple graph.

Multigraphs: Graphs that may have multiple edges connecting the same pair of vertices are called multigraphs.

Pseudographs: Graphs that may include loops, and possibly multiple edges connecting the same pair of vertices or a vertex to itself, are called pseudographs.

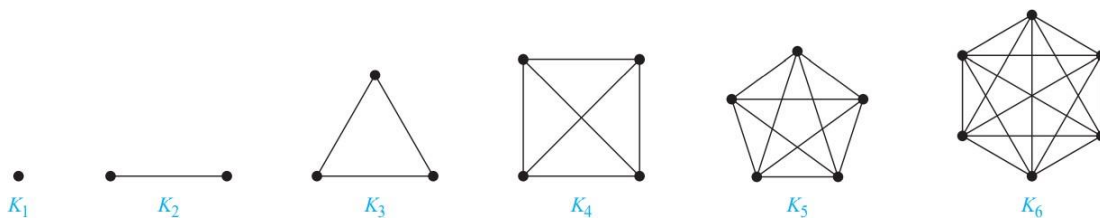
Null graph: A graph whose edge set is empty is called a null graph. In other words, a null graph does not contain any edges in it.

Trivial graph: A graph having only one vertex in it is called a trivial graph.

Some Special Simple Graphs

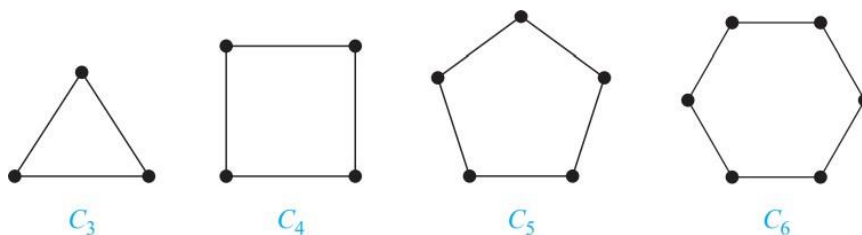
Complete Graphs: A complete graph on n vertices, denoted by K_n , is a simple graph that contains exactly one edge between each pair of distinct vertices.

The graphs K_n , for $n = 1, 2, 3, 4, 5, 6$, are displayed in Figure.

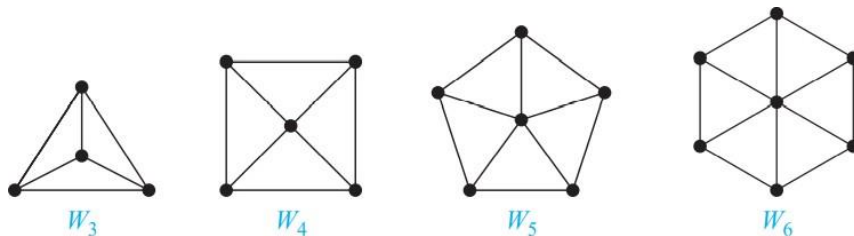


Non complete graph: A simple graph for which there is at least one pair of distinct vertices not connected by an edge is called noncomplete.

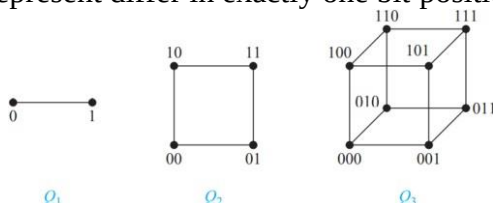
Cycles: A cycle C_n , $n \geq 3$, consists of n vertices $v_1, v_2, \dots, v_n$ and edges $\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{n-1}, v_n\}$, and $\{v_n, v_1\}$. The cycles C_3, C_4, C_5 , and C_6 are displayed in Figure.



Wheels: A wheel W_n is obtained when we add an additional vertex to a cycle C_n , for $n \geq 3$, and connect this new vertex to each of the n vertices in C_n by new edges.



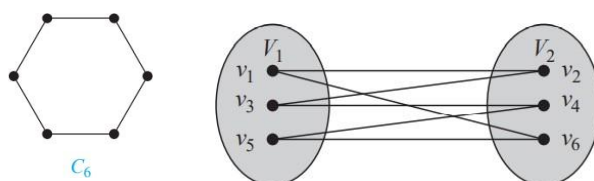
n- Cubes: An n -dimensional hypercube, or n -cube, denoted by Q_n , is a graph that has vertices representing the 2^n bit strings of length n . Two vertices are adjacent if and only if the bit strings that they represent differ in exactly one bit position.



Bipartite Graphs:

A simple graph G is called bipartite if its vertex set V can be partitioned into two disjoint sets V_1 and V_2 such that every edge in the graph connects a vertex in V_1 and a vertex in V_2 (so that no edge in G connects either two vertices in V_1 or two vertices in V_2). When this condition holds, we call the pair (V_1, V_2) a bipartition of the vertex set V of G .

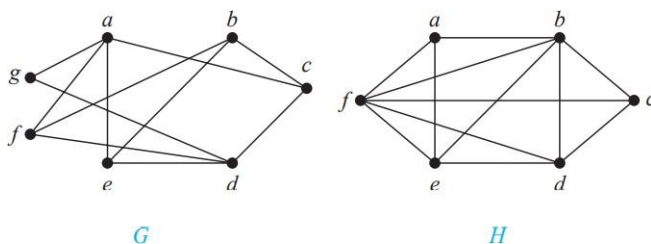
Example: C_6 is bipartite, as shown in Figure,



Because its vertex set can be partitioned into the two sets $V_1 = \{v_1, v_3, v_5\}$ and $V_2 = \{v_2, v_4, v_6\}$, and every edge of C_6 connects a vertex in V_1 and a vertex in V_2 .

Example: K_3 is not bipartite. To verify this, note that if we divide the vertex set of K_3 into two disjoint sets, one of the two sets must contain two vertices. If the graph were bipartite, these two vertices could not be connected by an edge, but in K_3 each vertex is connected to every other vertex by an edge.

Example: Are the graphs G and H displayed in Figure are bipartite?



Solution:

Graph G is bipartite because its vertex set is the union of two disjoint sets, $\{a, b, d\}$ and $\{c, e, f, g\}$, and each edge connects a vertex in one of these subsets to a vertex in the other subset. (Note that for G to be bipartite it is not necessary that every vertex in $\{a, b, d\}$ be adjacent to every vertex in $\{c, e, f, g\}$. For instance, b and g are not adjacent.)

Graph H is not bipartite because its vertex set cannot be partitioned into two subsets so that edges do not connect two vertices from the same subset. (verify this by considering the vertices a, b, and f.)

Theorem: A simple graph is bipartite if and only if it is possible to assign one of two different colors to each vertex of the graph so that no two adjacent vertices are assigned the same color.

Proof:

First, suppose that $G = (V, E)$ is a bipartite simple graph.

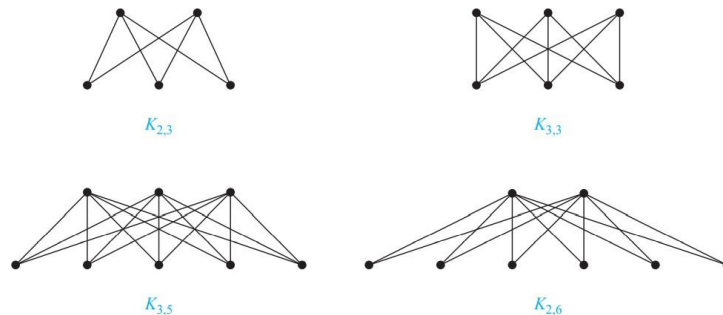
Then $V = V_1 \cup V_2$, where V_1 and V_2 are disjoint sets and every edge in E connects a vertex in V_1 and a vertex in V_2 .

If we assign one color to each vertex in V_1 and a second color to each vertex in V_2 , then no two adjacent vertices are assigned the same color. Now suppose that it is possible to assign colors to the vertices of the graph using just two colors so that no two adjacent vertices are assigned the same color.

Let V_1 be the set of vertices assigned one color and V_2 be the set of vertices assigned the other color. Then, V_1 and V_2 are disjoint and $V = V_1 \cup V_2$. Furthermore, every edge connects a vertex in V_1 and a vertex in V_2 because no two adjacent vertices are either both in V_1 or both in V_2 . Consequently, G is bipartite.

Complete Bipartite Graphs:

A complete bipartite graph $K_{m,n}$ is a graph that has its vertex set partitioned into two subsets of m and n vertices, respectively with an edge between two vertices if and only if one vertex is in the first subset and the other vertex is in the second subset. The complete bipartite graphs $K_{2,3}$, $K_{3,3}$, $K_{3,5}$, and $K_{2,6}$ are displayed in Figure.



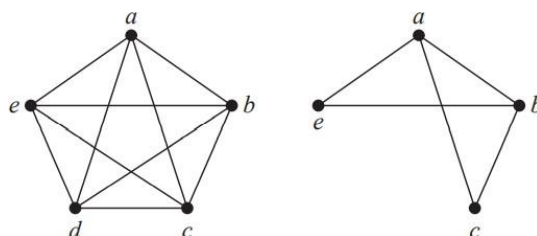
Sub graph:

A sub graph of a graph $G = (V, E)$ is a graph $H = (W, F)$, where $W \subseteq V$ and $F \subseteq E$. A sub graph H of G is a proper sub graph of G if $H \neq G$.

Given a set of vertices of a graph, we can form a sub graph of this graph with these vertices and the edges of the graph that connect them.

Let $G = (V, E)$ be a simple graph. The sub graph induced by a subset W of the vertex set V is the graph (W, F) , where the edge set F contains an edge in E if and only if both endpoints of this edge are in W .

Example: The graph G shown in Figure is a sub graph of K_5 . If we add the edge connecting c and e to G , we obtain the sub graph induced by $W = \{a, b, c, e\}$.



Acyclic graph:

A graph not containing any cycle in it is called as an acyclic graph.

Union of two simple graphs:

The union of two simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is the simple graph with vertex set $V_1 \cup V_2$ and edge set $E_1 \cup E_2$. The union of G_1 and G_2 is denoted by $G_1 \cup G_2$.

Removing Or Adding Edges Of A Graph:

Given a graph $G = (V, E)$ and an edge $e \in E$, we can produce a subgraph of G by removing the edge e . The resulting subgraph, denoted by $G - e$, has the same vertex set V as G . Its edge set is $E - e$. Hence, $G - e = (V, E - \{e\})$. Similarly, if E' is a subset of E , we can produce a subgraph of G by removing the edges in E' from the graph. The resulting subgraph has the same vertex set V as G . Its edge set is $E - E'$.

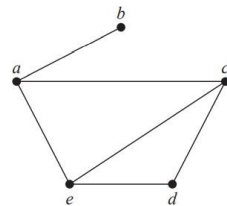
Representing Graphs and Graph Isomorphism

Introduction: There are many useful ways to represent graphs. In working with a graph it is helpful to be able to choose its most convenient representation.

Representing Graphs:

One way to represent a graph without multiple edges is to list all the edges of this graph. Another way to represent a graph with no multiple edges is to use adjacency lists, which specify the vertices that are adjacent to each vertex of the graph.

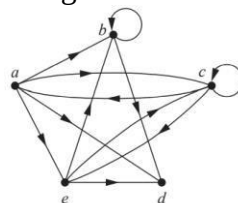
Example 1: Use adjacency lists to describe the simple graph given in Figure.



Solution: Table 1 lists those vertices adjacent to each of the vertices of the graph.

TABLE 1 An Adjacency List for a Simple Graph.	
Vertex	Adjacent Vertices
a	b, c, e
b	a
c	a, d, e
d	c, e
e	a, c, d

Example 2: Represent the directed graph shown in Figure by listing all the vertices that are the terminal vertices of edges starting at each vertex of the graph.



Solution: Table 2 represents the directed graph shown in Figure

TABLE 2 An Adjacency List for a Directed Graph.	
Initial Vertex	Terminal Vertices
<i>a</i>	<i>b, c, d, e</i>
<i>b</i>	<i>b, d</i>
<i>c</i>	<i>a, c, e</i>
<i>d</i>	
<i>e</i>	<i>b, c, d</i>

Adjacency Matrices:

Two types of matrices commonly used to represent graphs will be presented here. One is based on the adjacency of vertices, and the other is based on incidence of vertices and edges.

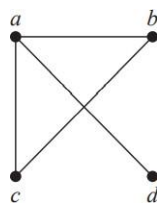
Adjacency matrix of undirected graph:

Let $G = (V, E)$ is a simple graph where $|V| = n$. Suppose that the vertices of G are listed arbitrarily as $v_1, v_2, \dots, v_n$. The adjacency matrix A (or A_G) of G , with respect to this listing of the vertices, is the $n \times n$ zero-one matrix with 1 as its (i, j) th entry when v_i and v_j are adjacent, and 0 as its (i, j) th entry when they are not adjacent.

In other words, if its adjacency matrix is $A = [a_{ij}]$, then

$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

Example 3: Use an adjacency matrix to represent the graph show in figure.



Solution: We order the vertices as a, b, c, d. The matrix representing this graph is

$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

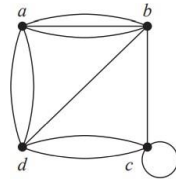
Example 4: Draw a graph with the adjacency matrix, with respect to the ordering of vertices a, b, c, d.

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

Solution: A graph with this adjacency matrix is shown in Figure below



Example 5: Use an adjacency matrix to represent the pseudograph shown in Figure.



Solution: The adjacency matrix using the ordering of vertices a, b, c, d is

$$\begin{bmatrix} 0 & 3 & 0 & 2 \\ 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 1 & 2 & 0 \end{bmatrix}.$$

Adjacency matrix to represent directed graphs:

The matrix for a directed graph $G = (V, E)$ has a 1 in its (i, j) th position if there is an edge from v_i to v_j , where $v_1, v_2, \dots, v_n$ is an arbitrary listing of the vertices of the directed graph. In other words, if $A = [a_{ij}]$ is the adjacency matrix for the directed graph with respect to this listing of the vertices, then

$$a_{ij} = \begin{cases} 1 & \text{if } (v_i, v_j) \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

The adjacency matrix for a directed graph does not have to be symmetric, because there may not be an edge from v_j to v_i when there is an edge from v_i to v_j .

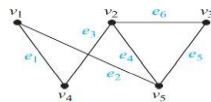
Adjacency matrices can also be used to represent directed multigraphs. Again, such matrices are not zero-one matrices when there are multiple edges in the same direction connecting two vertices. In the adjacency matrix for a directed multigraph, a_{ij} equals the number of edges that are associated to (v_i, v_j) .

Incidence Matrices:

Another common way to represent graphs is to use incidence matrices. Let $G = (V, E)$ be an undirected graph. Suppose that $v_1, v_2, \dots, v_n$ are the vertices and $e_1, e_2, \dots, e_m$ are the edges of G . Then the incidence matrix with respect to this ordering of V and E is the $n \times m$ matrix $M = [m_{ij}]$, where

$$m_{ij} = \begin{cases} 1 & \text{when edge } e_j \text{ is incident with } v_i, \\ 0 & \text{otherwise.} \end{cases}$$

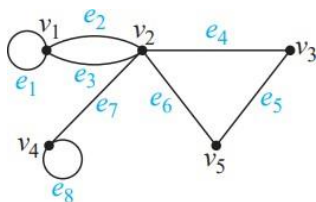
Example 6: Represent the graph shown in Figure with an incidence matrix.



Solution: The incidence matrix is,

$$\begin{matrix} & e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{matrix} & \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix} \end{matrix}.$$

Example 7 : Represent the pseudograph shown in Figure using an incidence matrix.



Solution: The incidence matrix for this graph is

$$\begin{matrix} & e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 & e_8 \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{matrix} & \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \end{matrix}.$$

Isomorphism of Graphs:

Definition:

The simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are *isomorphic* if there exists a one-to-one and onto function f from V_1 to V_2 with the property that a and b are adjacent in G_1 if and only if $f(a)$ and $f(b)$ are adjacent in G_2 , for all a and b in V_1 . Such a function f is called an *isomorphism*.* Two simple graphs that are not isomorphic are called *nonisomorphic*.

In other words, when two simple graphs are isomorphic, there is a one-to-one correspondence between vertices of the two graphs that preserves the adjacency relationship.

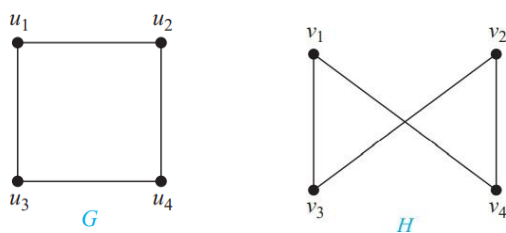
i.e both graphs satisfied

1) the number of vertices are equal

2) the number of edges are equal

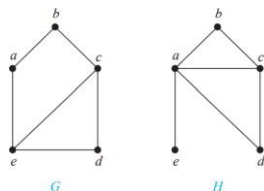
3) the sequence of degree of the vertices of both graphs are same.

Example 8: Show that the graphs $G = (V, E)$ and $H = (W, F)$, displayed in Figure, are isomorphic.



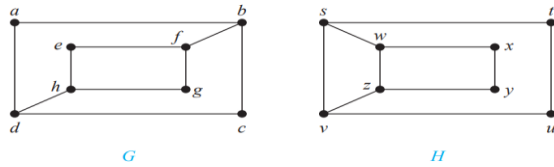
Solution: The function f with $f(u_1) = v_1$, $f(u_2) = v_4$, $f(u_3) = v_3$, and $f(u_4) = v_2$ is a one-to-one correspondence between V and W . To see that this correspondence preserves adjacency, note that adjacent vertices in G are u_1 and u_2 , u_1 and u_3 , u_2 and u_4 , and u_3 and u_4 , and each of the pairs $f(u_1) = v_1$ and $f(u_2) = v_4$, $f(u_1) = v_1$ and $f(u_3) = v_3$, $f(u_2) = v_4$ and $f(u_4) = v_2$, and $f(u_3) = v_3$ and $f(u_4) = v_2$ consists of two adjacent vertices in H .

Example 9: Show that the graphs displayed in Figure are not isomorphic.



Solution: Both G and H have five vertices and six edges. However, H has a vertex of degree one, namely, e , whereas G has no vertices of degree one. It follows that G and H are not isomorphic.

Example 10: Determine whether the graphs shown in Figure are isomorphic.



Solution: The graphs G and H both have eight vertices and 10 edges. They also both have four vertices of degree two and four of degree three. Because these invariants all agree, it is still conceivable that these graphs are isomorphic.

However, G and H are not isomorphic. To see this, note that because $\deg(a) = 2$ in G , a must correspond to either t, u, x , or y in H , because these are the vertices of degree two in H . However, each of these four vertices in H is adjacent to another vertex of degree two in H , which is not true for a in G .

Connectivity:

Walk: A walk is a sequence of edges that begins at a vertex of a graph and travels from vertex to vertex along edges of the graph.

Vertices and edges may be repeated in walk.

Open walk and closed walk:

A walk is said to be an open walk if the starting and ending vertices are different.

A walk is said to be a closed walk if the starting and ending vertices are same.

Trial: Trail is an open walk in which no edge is repeated but vertex can be repeated.

Circuit: A closed walk in which no edges are repeated.

Path: An open walk in which no vertex and edge appears more than once is called simple path or path.

Cycle: A cycle is a close path. That is, we start and end at the vertex.in the middle, we do not travel to any vertex twice. No edge is repeated in cycle.

Connected graph:

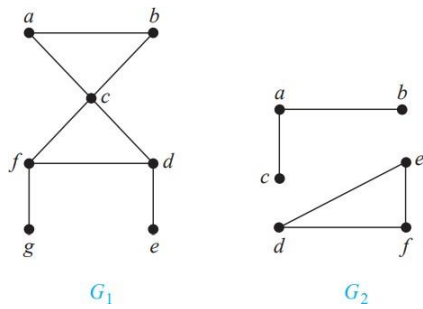
An undirected graph is called connected if there is a path between every pair of distinct vertices of the graph.

Disconnected graph:

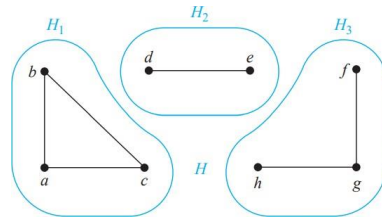
An undirected graph that is not connected is called disconnected.

We say that we disconnect a graph when we remove vertices or edges, or both, to produce a disconnected subgraph.

Examples: in the figure G_1 is connected and G_2 is disconnected



What are the connected components of the graph H shown in Figure



Solution: The graph H is the union of three disjoint connected subgraphs H_1 , H_2 , and H_3 , shown in Figure. These three subgraphs are the connected components of H.

Connected Components:

A connected component of a graph G is a connected subgraph of G that is not a proper subgraph of another connected subgraph of G . That is, a connected component of a graph G is a maximal connected subgraph of G .

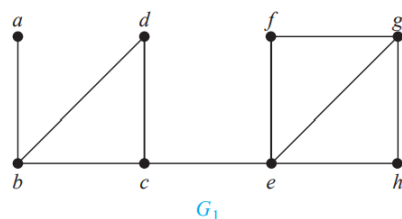
A graph G that is not connected has two or more connected components that are disjoint and have G as their union.

Cut vertices and cut edges:

Sometimes the removal from a graph of a vertex and all incident edges produces a subgraph with more connected components. Such vertices are called cut vertices (or articulation points). The removal of a cut vertex from a connected graph produces a subgraph that is not connected.

Analogously, an edge whose removal produces a graph with more connected components than in the original graph is called a cut edge or bridge.

Find the cut vertices and cut edges in the graph G_1 shown in Figure.



Solution: The cut vertices of G_1 are b , c , and e . The removal of one of these vertices (and its adjacent edges) disconnects the graph. The cut edges are $\{a, b\}$ and $\{c, e\}$. Removing either one of these edges disconnects G_1 .

Vertex Connectivity :

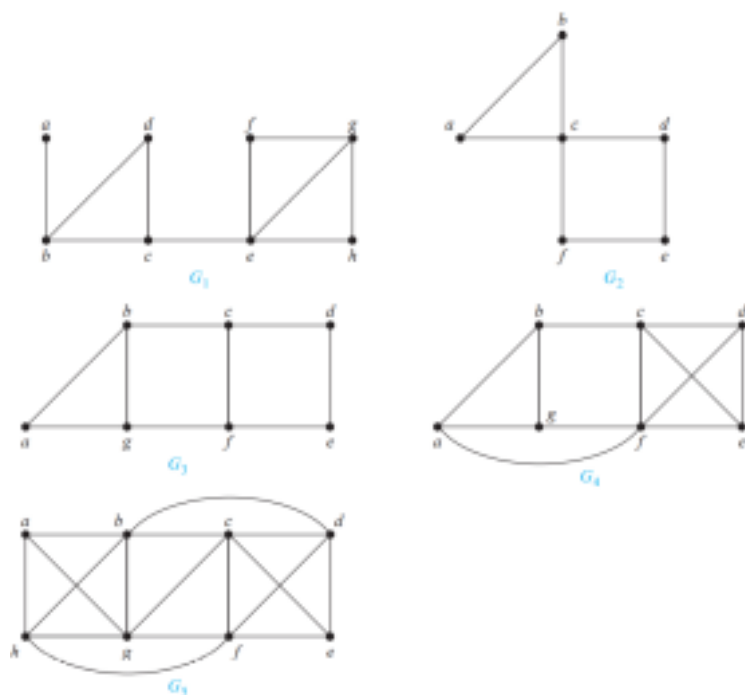
Not all graphs have cut vertices. For example, the complete graph K_n , where $n \geq 3$, has no cut vertices. When you remove a vertex from K_n and all edges incident to it, the resulting subgraph is the complete graph K_{n-1} , a connected graph.

Connected graphs without cut vertices are called **nonseparable** graphs, and can be thought of as more connected than those with a cut vertex.

Vertex connectivity of a noncomplete graph G , denoted by $\kappa(G)$, is the minimum number of vertices in a vertex cut.

When G is a complete graph, it has no vertex cuts, because removing any subset of its vertices and all incident edges still leaves a complete graph. Consequently, we cannot define $\kappa(G)$ as the minimum number of vertices in a vertex cut when G is complete. Instead, we set $\kappa(K_n) = n - 1$, the number of vertices needed to be removed to produce a graph with a single vertex.

Example: Find the vertex connectivity for each of the graphs in Figure.



Solution: Each of the five graphs in Figure is connected and has more than vertex, so each of these graphs has positive vertex connectivity. Because G_1 is a connected graph with a cut vertex, we know that $\kappa(G_1) = 1$. Similarly, $\kappa(G_2) = 1$, because c is a cut vertex of G_2 . G_3 has $\{b, g\}$ is a vertex cut. Hence, $\kappa(G_3) = 2$. Similarly, because G_4 has a vertex cut of size two, $\{c, f\}$. It follows that $\kappa(G_4) = 2$. Then G_5 has vertex cut of size three, $\{b, c, f\}$ is a vertex cut of G_5 . Hence, $\kappa(G_5) = 3$.

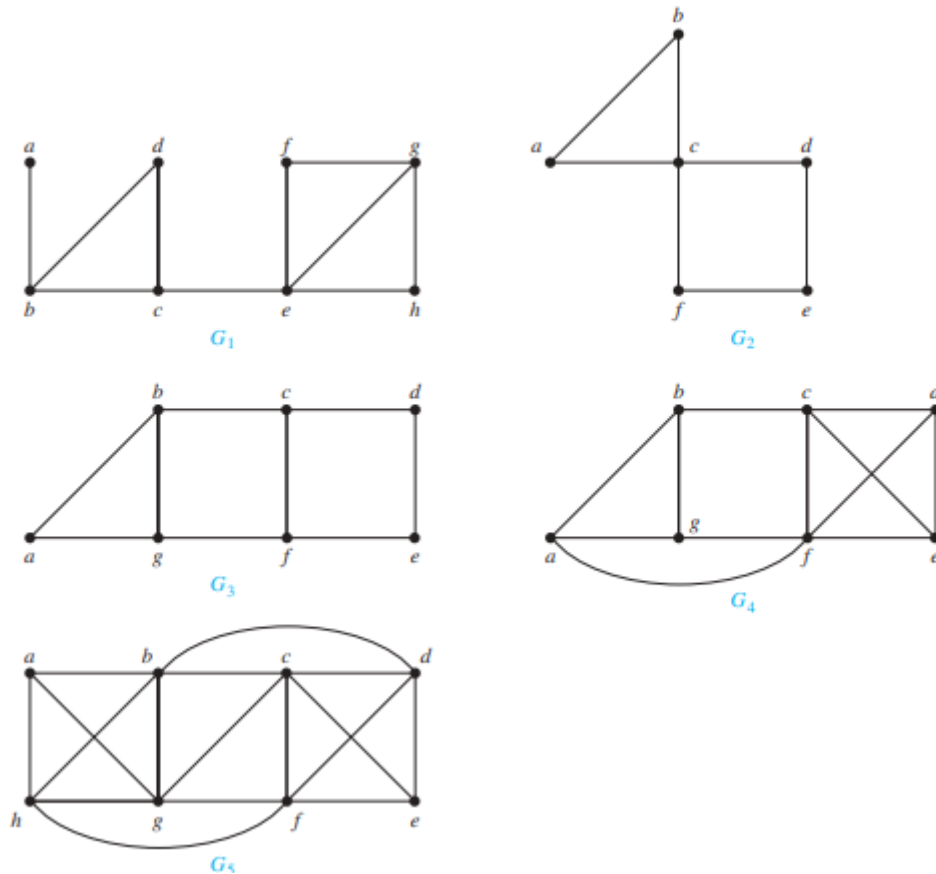
Edge Connectivity:

We can also measure the connectivity of a connected graph $G = (V, E)$ in terms of the minimum number of edges that we can remove to disconnect it. If a graph has a cut edge, then we need only remove it to disconnect G . If G does not have a cut edge, we look for the smallest set of edges that can be removed to disconnect it. A set of edges E' is called an edge cut of G if the subgraph $G - E'$ is disconnected.

The **edge connectivity** of a graph G , denoted by $\lambda(G)$, is the minimum number of edges in an edge cut of G .

This defines $\lambda(G)$ for all connected graphs with more than one vertex because it is always possible to disconnect such a graph by removing all edges incident to one of its vertices. Note that $\lambda(G) = 0$ if G is not connected.

Example: Find the edge connectivity of each of the graphs in Figure.



Solution: Each of the five graphs in Figure is connected and has more than one vertex, so we know that all of them have positive edge connectivity. G_1 has a cut edge, so $\lambda(G_1) = 1$

In graph G_2 the removal of the two edges $\{a, b\}$ and $\{a, c\}$ disconnects it. Hence, $\lambda(G_2) = 2$. Similarly, $\lambda(G_3) = 2$.

In G_3 the removal of the two edges $\{b, c\}$ and $\{f, g\}$ disconnects it. In G_4 , but the removal of the three edges $\{b, c\}$, $\{a, f\}$, and $\{f, g\}$ disconnects it. Hence, $\lambda(G_4) = 3$.

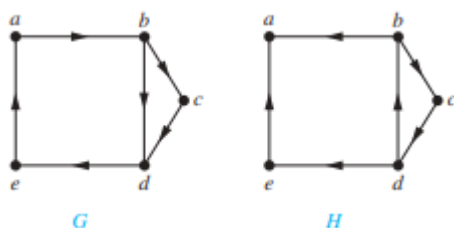
Finally, $\lambda(G_5) = 3$, because the removal of any two of its edges does not disconnect it, but the removal of $\{a, b\}$, $\{a, g\}$, and $\{a, h\}$ does.

Connectedness in Directed Graphs:

A directed graph is **strongly connected** if there is a path from a to b and from b to a whenever a and b are vertices in the graph.

A directed graph is **weakly connected** if there is a path between every two vertices in the underlying undirected graph.

Example: Are the directed graphs G and H shown in Figure strongly connected? Are they weakly connected?



Solution: G is strongly connected because there is a path between any two vertices in this directed graph. The graph H is not strongly connected. There is no directed path from a to b in this graph. However, H is weakly connected, because there is a path between any two vertices in the underlying undirected graph of H .

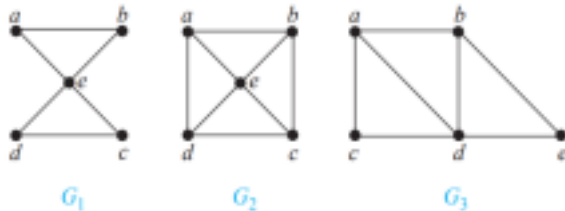
Euler and Hamilton Path:

Euler path and Euler circuit:

An **Euler circuit** in a graph G is a simple circuit containing every edge of G .

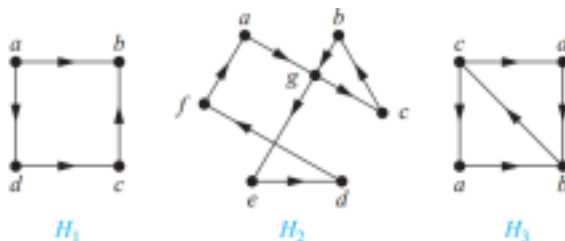
An **Euler path** in G is a simple path containing every edge of G .

Example: Which of the undirected graphs in Figure have an Euler circuit? Of those that do not, which have an Euler path?



Solution: The graph G_1 has an Euler circuit, for example, a, e, c, d, e, b, a . Neither of the graphs G_2 or G_3 has an Euler circuit. However, G_3 has an Euler path, namely, a, c, d, e, b, d, a, b . G_2 does not have an Euler path.

Example: Which of the directed graphs in Figure have an Euler circuit? Of those that do not, which have an Euler path?

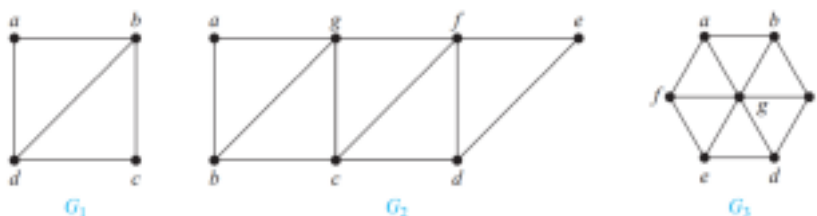


Solution: The graph H_2 has an Euler circuit, for example, $a, g, c, b, g, e, d, f, a$. Neither H_1 nor H_3 has an Euler circuit. H_3 has an Euler path, namely, c, a, b, c, d, b , but H_1 does not.

Necessary And Sufficient Conditions For Euler Circuits And Paths:

- A connected multigraph has an Euler path but not an Euler circuit if and only if it has exactly two vertices of odd degree.
- A connected multigraph has an Euler circuit if and only if each of its vertices has even degree.

Example: Which graphs shown in Figure have an Euler path?



Solution: G_1 contains exactly two vertices of odd degree, namely, b and d . Hence, it has an Euler path that must have b and d as its endpoints. One such Euler path is d, a, b, c, d, b . Similarly, G_2 has exactly two vertices of odd degree, namely, b and d . So it has an Euler path that must have b and d as endpoints. One such Euler path is $b, a, g, f, e, d, c, g, b, c, f, d$. G_3 has no Euler path because it has six vertices of odd degree.

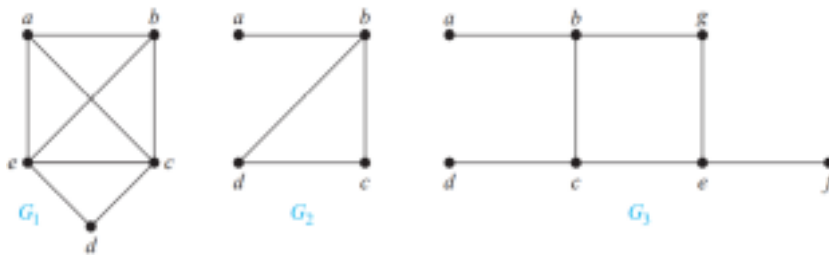
Hamilton Paths and Circuits:

A **simple path** in a graph G that passes through every vertex exactly once is called a

Hamilton path. A **simple circuit** in a graph G that passes through every vertex exactly once is called a **Hamilton circuit**.

Example: Which of the simple graphs in Figure have a Hamilton circuit or, if not, a Hamilton path?

Solution: G_1 has a Hamilton circuit: a, b, c, d, e, a . There is no Hamilton circuit in G_2 (this can be seen by noting that any circuit containing every vertex must contain the edge $\{a, b\}$ twice), but G_2 does have a Hamilton path, namely, a, b, c, d . G_3 has neither a Hamilton circuit nor a Hamilton path, because any path containing all vertices must contain one of the edges $\{a, b\}$, $\{e, f\}$, and $\{c, d\}$ more than once.

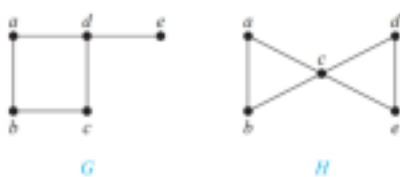


Conditions For The Existence Of Hamilton Circuits:

a graph with a vertex of degree one cannot have a Hamilton circuit, because in a Hamilton circuit, each vertex is incident with two edges in the circuit.

Also, note that when a Hamilton circuit is being constructed and this circuit has passed through a vertex, then all remaining edges incident with this vertex, other than the two used in the circuit, can be removed from consideration. Furthermore, a Hamilton circuit cannot contain a smaller circuit within it.

Example: Show that neither graph displayed in Figure has a Hamilton circuit.



Solution: There is no Hamilton circuit in G because G has a vertex of degree one, namely, e . Now consider H . Because the degrees of the vertices a, b, d , and e are all two, every edge incident with these vertices must be part of any Hamilton circuit. It is now easy to see that no Hamilton circuit can exist in H , for any Hamilton circuit would have to contain four edges incident with c , which is impossible.

Example: Show that K_n has a Hamilton circuit whenever $n \geq 3$.

Solution: We can form a Hamilton circuit in K_n beginning at any vertex. Such a circuit can be built by visiting vertices in any order we choose, as long as the path begins and ends at the same vertex and visits each other vertex exactly once. This is possible because there are edges in K_n between any two vertices.

Weighted graphs:

Graphs that have a number assigned to each edge are called weighted graphs.

The length of a path in a weighted graph be the sum of the weights of the edges of this path.

Shortest Path from a Specified Vertex to another Specified Vertex

The problem of finding the shortest path from a specified vertex 'a' to another specified vertex 'z' (here showing only 'a' to 'z', but it's also applicable for any vertex to another vertex will be same techniques):

Dijkstra's Algorithm

Description of the Algorithm

Dijkstra's algorithms labels the vertices of the given digraph. At each stage in the algorithm some vertices have permanent labels and others temporary labels,

The algorithm begins by assigning a permanent label 0 to the starting vertex a, and a temporary label ∞ to the remaining $n - 1$ vertices. From then on, in each iteration another vertex gets a permanent label, according to the following rules:

Step 1: Every vertex j that is not yet permanently labelled gets a new temporary label whose value is given by
 $\min(\text{old label of } j, (\text{old label of } i + d_{ij}))$

Where i is the latest vertex permanently labelled, in the previous iteration, and d_{ij} is the direct distance between vertices i and j . If i and j are not joined by an edge, then $d_{ij} = \infty$

Step 2: The smallest value among all the temporary labels is found, and this becomes the permanent label of the corresponding vertex. In case of a tie, select any one of the candidates for permanent labelling.

Steps 1 and 2 are repeated alternately until the destination vertex z gets a permanent label. The first vertex to be permanently labelled is at a distance of zero from a .

The second vertex to get a permanent label (out of the remaining $n - 1$ vertices) is the vertex closest to a .

From the remaining $n - 2$ vertices, the next one to be permanently labelled is the second closest vertex to a . And so on. The permanent label of each vertex is the shortest distance of that vertex from a .

The permanent labels will be shown enclosed in a square, and the most recently assigned permanent label in the vector is indicated by a tick.

Planar graph:

A graph is called planar if it can be drawn in the plane without any edges crossing (where a crossing of edges is the intersection of the lines or arcs representing them at a point other than their common endpoint). Such a drawing is called a planar representation of the graph.

Region: A planar representation of a graph splits the plane into regions, including an unbounded region.

Euler's Formula: Let G be a connected planar simple graph with e edges and v vertices. Let r be the number of regions in a planar representation of G . Then $r = e - v + 2$

Example: Suppose that a connected planar simple graph has 20 vertices, each of degree 3. Into how many regions does a representation of this planar graph split the plane?

Solution: This graph has 20 vertices, each of degree 3, $v = 20$. But we know that the sum of the degrees of the vertices, is equal to twice the number of edges,

$$\begin{aligned}\sum \deg(v) &= 2e \\ 20 \times 3 &= 2e \\ 2e &= 60 \\ e &= 30 \text{ or}\end{aligned}$$

Consequently, from Euler's formula, the number of regions is $r = e - v + 2$
 $r = 30 - 20 + 2 = 12$ regions.

Graph Coloring:

Problems related to the coloring of maps of regions, such as maps of parts of the world, have generated many results in graph theory. When a map is colored, two regions with a common border are customarily assigned different colors. One way to ensure that two adjacent regions never have the same color is to use a different color for each region. However, this is inefficient, and on maps with many regions it would be hard to distinguish similar colors. Instead, a small number of colors should be used whenever possible.

A coloring of a simple graph is the assignment of a color to each vertex of the graph so that no two adjacent vertices are assigned the same color.

The chromatic number of a graph is the least number of colors needed for a coloring of this graph. The chromatic number of a graph G is denoted by $\chi(G)$. (Here χ is the Greek letter chi.)

Color class: A color class is set of vertices of graph which are having the same coloring is done in graph.

Properties of Chromatic Numbers

1. A graph which is totally disconnected, has isolated vertices. No two vertices are adjacent. Therefore the graph is 1 - chromatic.
2. A graph containing one edge or more edges is at least 2 - chromatic
3. A cycle with 3 vertices is 3 - chromatic
4. A complete graph with p - vertices is p - chromatic
5. A graph which is a cycle of $2n$ points is 2 - chromatic
6. A graph which is a cycle of $2n + 1$ points is 3 - chromatic
7. If W_n is a wheel, having one vertex at the centre and $n - 1$ vertices along the circumference
8. $\chi(W_n) = 4$ if n is even and $\chi(W_n) = 3$ if n is odd integer.

Example: What is the chromatic number of K_n

Solution

Coloring of K_n can be constructed using ' n ' colors by assigning a different color to each vertex. No two vertices can be assigned the same color, because every two vertices of this graph are adjacent. Hence, the chromatic number of K_n is n . That is, $\chi(K_n) = n$.

Example: What is the chromatic number of the complete bipartite graph K where m and n are positive integers?

Solution

The number of colors needed may seem to depend on m and n .

Only two colors are needed, because K , is a bipartite graph.

Hence, $\chi(K_{m,n}) = 2$.

This means that we can color the set of m vertices with one color and the set of n vertices with a second color. Because edges connect only a vertex from the set of m vertices and a vertex from the set of n vertices, no two adjacent vertices have the same color.

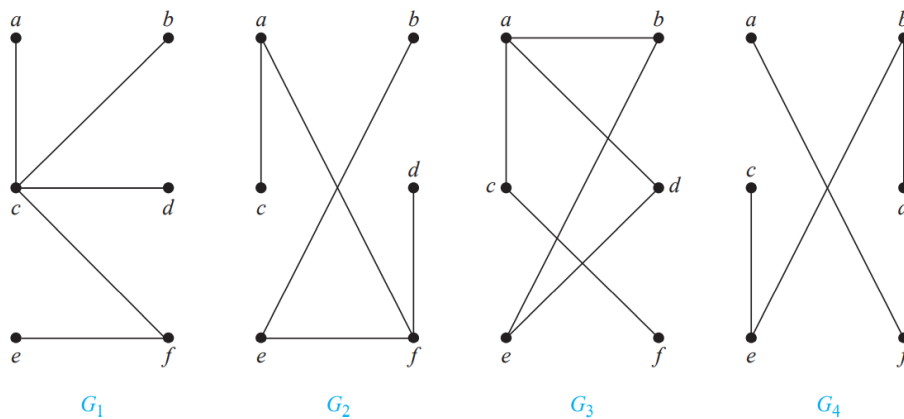
TREES

Tree:

A tree is a connected undirected graph with no simple circuits. Because a tree cannot have a simple circuit, a tree cannot contain multiple edges or loops. Therefore any tree must be a simple graph.

Example: Which of the graphs shown in Figure are trees?

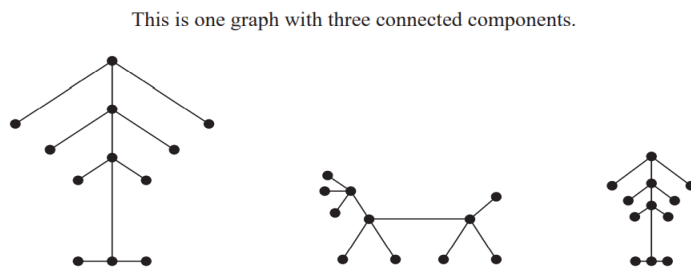
Solution: G_1 and G_2 are trees, because both are connected graphs with no simple circuits. G_3 is not a tree because e, b, a, d, e is a simple circuit in this graph. Finally, G_4 is not a tree because it is not connected.



Any connected graph that contains no simple circuits is a **tree**.

Graphs containing no simple circuits that are not connected these graphs are called **forests** and have the property that each of their connected components is a tree.

Example for forest:



This is one graph with three connected components.

Graph G

Trees are often defined as undirected graphs with the property that there is a unique simple path between every pair of vertices.

Rooted trees:

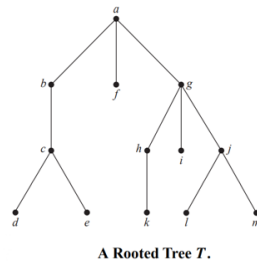
A rooted tree is a tree in which one vertex has been designated as the root and every edge is directed away from the root.

The terminology for trees has botanical and genealogical origins. Suppose that T is a rooted tree. If v is a vertex in T other than the root, the **parent** of v is the unique vertex u such that there is a directed edge from u to v . When u is the **parent** of v , v is called a **child** of u . Vertices with the **same parent** are called **siblings**. The **ancestors** of a vertex other than the root are the vertices in the path from the root to this vertex, excluding the vertex itself and including the root (that is, its parent, its parent's parent, and so on, until the root is reached). The **descendants** of a vertex v are those vertices that have v as an **ancestor**.

A vertex of a rooted tree is called a **leaf** if it has **no children**. Vertices that have children are called **internal vertices**. The root is an internal vertex unless it is the only vertex in the graph, in which case it is a leaf.

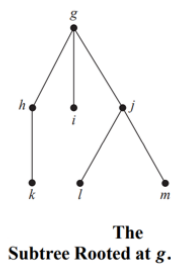
If a is a vertex in a tree, the **subtree** with a as its root is the **subgraph** of the tree consisting of a and its descendants and all edges incident to these descendants.

Example: In the rooted tree T (with root a) shown in Figure, find the parent of c , the children of g , the siblings of h , all ancestors of e , all descendants of b , all internal vertices, and all leaves. What is the subtree rooted at g ?



Solution: The parent of c is b . The children of g are h , i , and j . The siblings of h are i and j . The ancestors of e are c , b , and a . The descendants of b are c , d , and e . The internal vertices are a , b , c , g , h , and j . The leaves are d , e , f , i , k , l , and m .

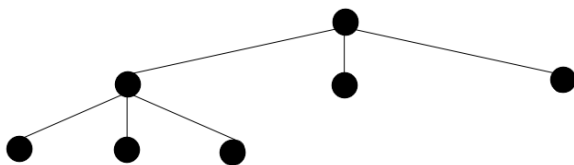
The subtree rooted at g is shown in Figure



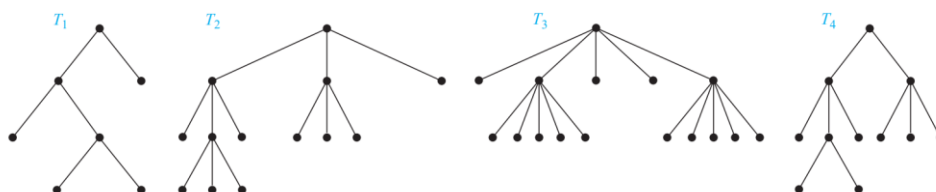
m-ary tree:

A rooted tree is called an *m-ary tree* if every internal vertex has no more than m children. The tree is called a *full m-ary tree* if every internal vertex has exactly m children. An *m-ary tree* with $m = 2$ is called a *binary tree*.

Example of a full 3-ary tree:



Example: Are the rooted trees in Figure full m -ary trees for some positive integer m ?



Solution: T_1 is a full binary tree because each of its internal vertices has two children. T_2 is a full 3-ary tree because each of its internal vertices has three children. In T_3 each internal vertex has five children, so T_3 is a full 5-ary tree.

T_4 is not a full m -ary tree for any m because some of its internal vertices have two children and others have three children.

Trees as Models:

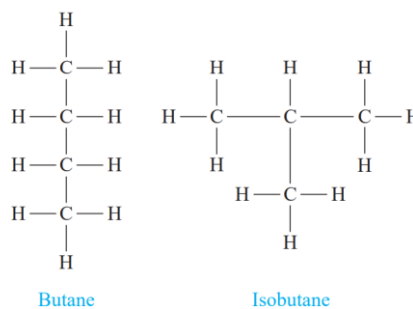
Trees are used as models in such diverse areas as computer science, chemistry, geology, botany, and psychology. We will describe a variety of such models based on trees.

Example: Saturated Hydrocarbons and Trees:

Graphs can be used to represent molecules, where atoms are represented by vertices and bonds between them by edges. The English mathematician Arthur Cayley discovered trees in 1857 when he was trying to enumerate the isomers of compounds of the form C_nH_{2n+2} , which are called saturated hydrocarbons.

In graph models of saturated hydrocarbons, each carbon atom is represented by a vertex of degree 4, and each hydrogen atom is represented by a vertex of degree 1. There are $3n + 2$ vertices in a graph representing a compound of the form C_nH_{2n+2} . The number of edges in such a graph is half the sum of the degrees of the vertices. Hence, there are $(4n + 2n + 2)/2 = 3n + 1$ edges in this graph. Because the graph is connected and the number of edges is one less than the number of vertices, it must be a tree.

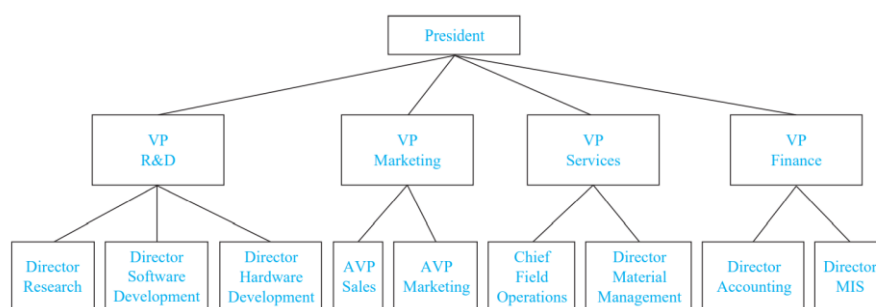
The nonisomorphic trees with n vertices of degree 4 and $2n + 2$ of degree 1 represent the different isomers of C_nH_{2n+2} . For instance, when $n = 4$, there are exactly two nonisomorphic trees of this type. Hence, there are exactly two different isomers of C_4H_{10} . Their structures are displayed in Figure. These two isomers are called butane and isobutane.



The Two Isomers of Butane.

Example: Representing Organizations:

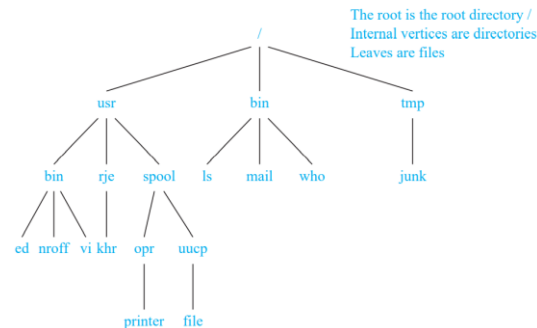
The structure of a large organization can be modeled using a rooted tree. Each vertex in this tree represents a position in the organization. An edge from one vertex to another indicates that the person represented by the initial vertex is the (direct) boss of the person represented by the terminal vertex. The graph shown in Figure displays such a tree. In the organization represented by this tree, the Director of Hardware Development works directly for the Vice President of R&D.



An Organizational Tree for a Computer Company.

Example: Computer File Systems:

Files in computer memory can be organized into directories. A directory can contain both files and subdirectories. The root directory contains the entire file system. Thus, a file system may be represented by a rooted tree, where the root represents the root directory, internal vertices represent subdirectories, and leaves represent ordinary files or empty directories. One such file system is shown in Figure. In this system, the file *kh* is in the directory *rje*.

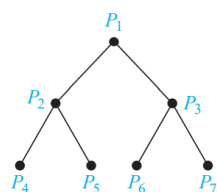


A Computer File System.

Example: Tree-Connected Parallel Processors :

A tree-connected network is another important way to interconnect processors. The graph representing such a network is a complete binary tree, that is, a full binary tree where every root is at the same level. Such a network interconnects $n = 2k - 1$ processors, where k is a positive integer. A processor represented by the vertex v that is not a root or a leaf has three two-way connections—one to the processor represented by the parent of v and two to the processors represented by the two children of v . The processor represented by the root has two two-way connections to the processors represented by its two children. A processor represented by a leaf v has a single two-way connection to the parent of v . We display a tree-connected network with seven processors.

We now illustrate how a tree-connected network can be used for parallel computation. In particular, we show how the processors in Figure can be used to add eight numbers, using three steps.



A
Tree-Connected
Network of Seven
Processors.

In the first step, we add x_1 and x_2 using P_4 , x_3 and x_4 using P_5 , x_5 and x_6 using P_6 and x_7 and x_8 using P_7 . In the second step, we add $x_1 + x_2$ and $x_3 + x_4$ using P_2 and $x_5 + x_6$ and $x_7 + x_8$ using P_3 . Finally, in the third step, we add $x_1 + x_2 + x_3 + x_4$ and $x_5 + x_6 + x_7 + x_8$ using P_1 . The three steps used to add eight numbers compares favorably to the seven steps required to add eight numbers serially, where the steps are the addition of one number to the sum of the previous numbers in the list.

Tree Traversal:

Universal Address Systems:

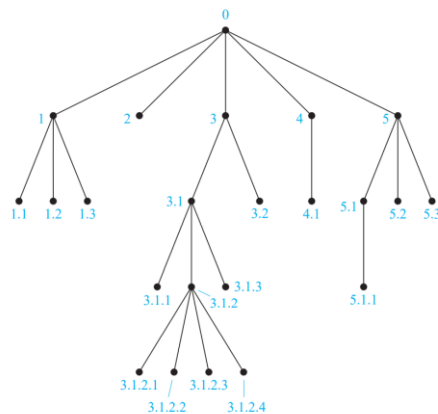
Procedures for traversing all vertices of an ordered rooted tree rely on the orderings of children. In ordered rooted trees, the children of an internal vertex are shown from left to right in the drawings representing these directed graphs.

We will describe one way we can totally order the vertices of an ordered rooted tree. To produce this ordering, we must first label all the vertices. We do this recursively:

1. Label the root with the integer 0. Then label its k children (at level 1) from left to right with 1, 2, 3,..., k .
2. For each vertex v at level n with label A , label its k_v children, as they are drawn from left to right, with $A.1, A.2, \dots, A.k_v$.

Following this procedure, a vertex v at level n , for $n \geq 1$, is labeled $x_1.x_2. \dots .x_n$, where the unique path from the root to v goes through the x_1 st vertex at level 1, the x_2 nd vertex at level 2, and so on. This labeling is called the universal address system of the ordered rooted tree.

Example: Display the labelings of the universal address system next to the vertices in the ordered rooted tree shown in Figure.



The Universal Address System of an Ordered Rooted Tree.

The lexicographic ordering of the labelings is

$0 < 1 < 1.1 < 1.2 < 1.3 < 2 < 3 < 3.1 < 3.1.1 < 3.1.2 < 3.1.2.1 < 3.1.2.2 < 3.1.2.3 < 3.1.2.4 < 3.1.3 < 3.2 < 4 < 4.1 < 5 < 5.1 < 5.1.1 < 5.2 < 5.3$.

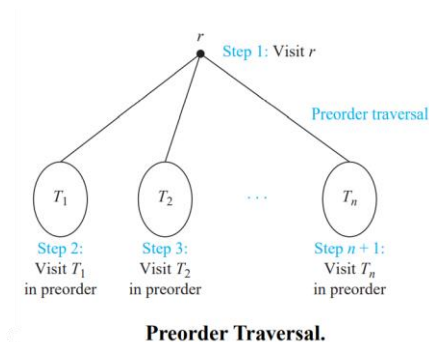
Traversal Algorithms :

Procedures for systematically visiting every vertex of an ordered rooted tree are called traversal algorithms. We will describe three of the most commonly used such algorithms, preorder traversal, inorder traversal, and postorder traversal.

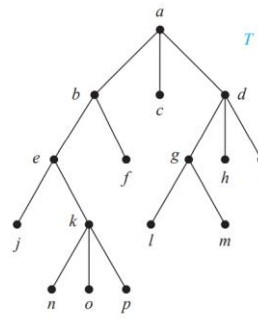
Preorder traversal:

Let T be an ordered rooted tree with root r . If T consists only of r , then r is the *preorder traversal* of T . Otherwise, suppose that $T_1, T_2, \dots, T_n$ are the subtrees at r from left to right in T . The *preorder traversal* begins by visiting r . It continues by traversing T_1 in preorder, then T_2 in preorder, and so on, until T_n is traversed in preorder.

Below figure indicates how a preorder traversal is carried out:



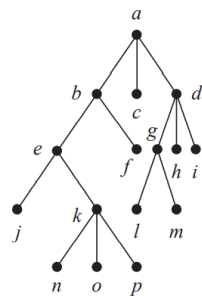
Example: In which order does a preorder traversal visit the vertices in the ordered rooted tree T shown in Figure?



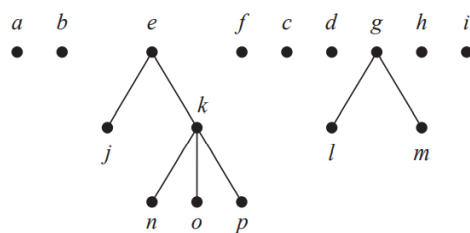
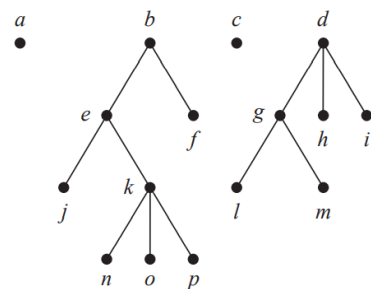
**The Ordered
Rooted Tree T .**

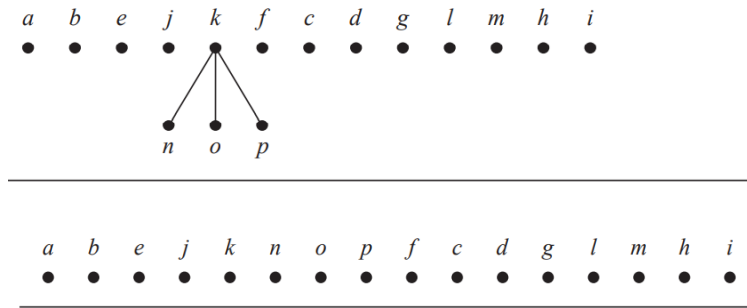
Solution: The steps of the preorder traversal of T are shown in Figure. We traverse T in preorder by first listing the root a , followed by the preorder list of the subtree with root b , the preorder list of the subtree with root c (which is just c) and the preorder list of the subtree with root d .

The preorder list of the subtree with root b begins by listing b , then the vertices of the subtree with root e in preorder, and then the subtree with root f in preorder (which is just f). The preorder list of the subtree with root d begins by listing d , followed by the preorder list of the subtree with root g , followed by the subtree with root h (which is just h), followed by the subtree with root i (which is just i). The preorder list of the subtree with root e begins by listing e , followed by the preorder listing of the subtree with root j (which is just j), followed by the preorder listing of the subtree with root k . The preorder listing of the subtree with root g is g followed by l , followed by m . The preorder listing of the subtree with root k is k , n , o , p . Consequently, the preorder traversal of T is $a, b, e, j, k, n, o, p, f, c, d, g, l, m, h, i$.



Preorder traversal: Visit root,
visit subtrees left to right



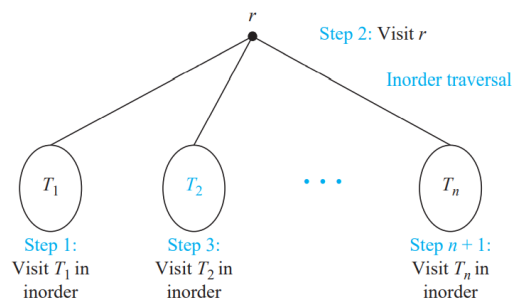


The Preorder Traversal of T .

Inorder traversal:

Let T be an ordered rooted tree with root r . If T consists only of r , then r is the *inorder traversal* of T . Otherwise, suppose that $T_1, T_2, \dots, T_n$ are the subtrees at r from left to right. The *inorder traversal* begins by traversing T_1 in inorder, then visiting r . It continues by traversing T_2 in inorder, then T_3 in inorder, $\dots$, and finally T_n in inorder.

Below figure indicates how inorder traversal is carried out.



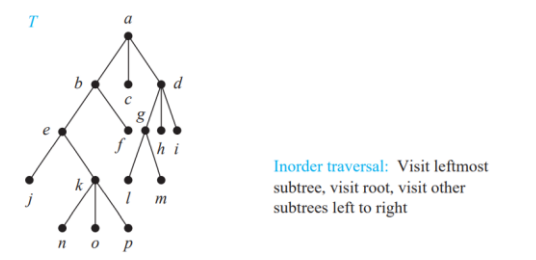
Inorder Traversal.

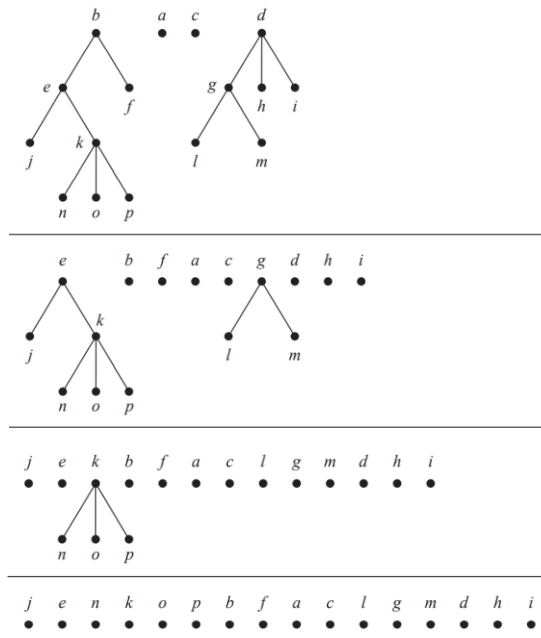
Example: In which order does an inorder traversal visit the vertices of the ordered rooted tree T in Figure?

Solution: The steps of the inorder traversal of the ordered rooted tree T are shown in Figure. The inorder traversal begins with an inorder traversal of the subtree with root b , the root a , the inorder listing of the subtree with root c , which is just c , and the inorder listing of the subtree with root d .

The inorder listing of the subtree with root b begins with the inorder listing of the subtree with root e , the root b , and f . The inorder listing of the subtree with root d begins with the inorder listing of the subtree with root g , followed by the root d , followed by h , followed by i . The inorder listing of the subtree with root e is j , followed by the root e , followed by the inorder listing of the subtree with root k . The inorder listing of the subtree with root g is l , g , m . The inorder listing of the subtree with root k is n , k , o , p .

Consequently, the inorder listing of the ordered rooted tree is $j, e, n, k, o, p, b, f, a, c, l, g, m, d, h, i$.



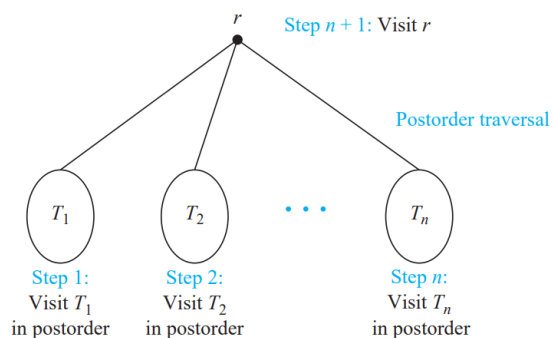


The Inorder Traversal of T .

Postorder traversal:

Let T be an ordered rooted tree with root r . If T consists only of r , then r is the *postorder traversal* of T . Otherwise, suppose that $T_1, T_2, \dots, T_n$ are the subtrees at r from left to right. The *postorder traversal* begins by traversing T_1 in postorder, then T_2 in postorder, $\dots$, then T_n in postorder, and ends by visiting r .

Below figure illustrates how postorder traversal is done.



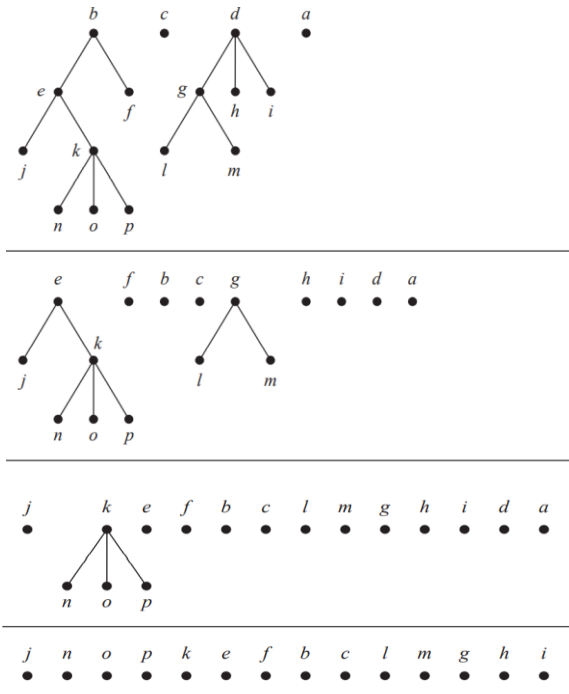
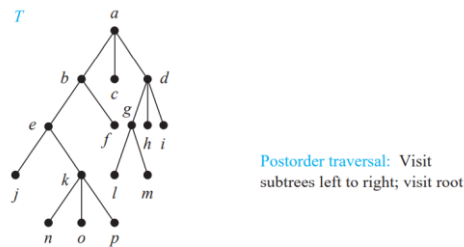
Example: In which order does a postorder traversal visit the vertices of the ordered rooted tree T shown in Figure?

Solution: The steps of the postorder traversal of the ordered rooted tree T are shown in Figure.

The postorder traversal begins with the postorder traversal of the subtree with root b , the postorder traversal of the subtree with root c , which is just c , the postorder traversal of the subtree with root d , followed by the root a .

The postorder traversal of the subtree with root b begins with the postorder traversal of the subtree with root e , followed by f , followed by the root b . The postorder traversal of the rooted tree with root d begins with the postorder traversal of the subtree with root g , followed by h , followed by i , followed by the root d . The postorder traversal of the subtree with root e begins with j , followed by the postorder traversal of the subtree with root k , followed by the root e . The postorder traversal of the subtree with root g is l, m, g . The postorder traversal of the subtree with root k is n, o, p, k .

Therefore, the postorder traversal of T is $j, n, o, p, k, e, f, b, c, l, m, g, h, i, d, a$.



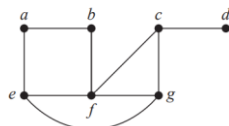
The Postorder Traversal of T .

Spanning Trees:

Let G be a simple graph. A *spanning tree* of G is a subgraph of G that is a tree containing every vertex of G .

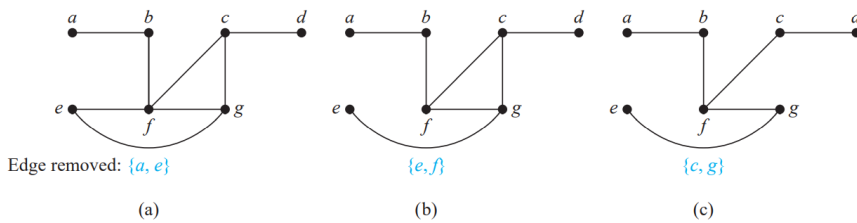
A simple graph with a spanning tree must be connected, because there is a path in the spanning tree between any two vertices. The converse is also true; that is, every connected simple graph has a spanning tree.

Example: Find a spanning tree of the simple graph G shown in Figure.



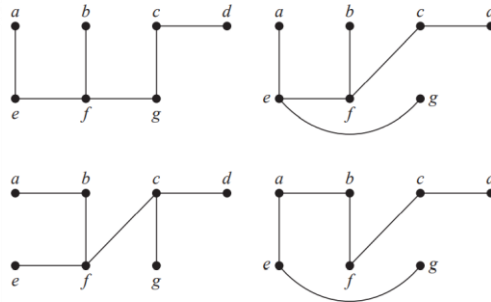
The Simple Graph G .

Solution: The graph G is connected, but it is not a tree because it contains simple circuits. Remove the edge $\{a, e\}$. This eliminates one simple circuit, and the resulting subgraph is still connected and still contains every vertex of G . Next remove the edge $\{e, f\}$ to eliminate a second simple circuit. Finally, remove edge $\{c, g\}$ to produce a simple graph with no simple circuits. This subgraph is a spanning tree, because it is a tree that contains every vertex of G . The sequence of edge removals used to produce the spanning tree is illustrated in Figure.



Producing a Spanning Tree for G by Removing Edges That Form Simple Circuits.

Other possible spanning trees:



Spanning Trees of G .

Minimum Spanning Trees:

A *minimum spanning tree* in a connected weighted graph is a spanning tree that has the smallest possible sum of weights of its edges.

We will discuss two algorithms for constructing minimum spanning trees.

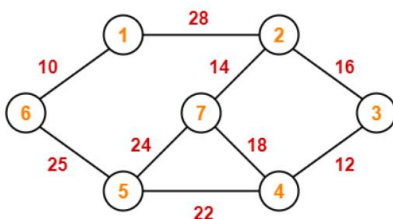
Prim's algorithm:

The first algorithm that we will discuss was originally discovered by the Czech mathematician Vojtech Jarník in 1930, who described it in a paper in an obscure Czech journal. The algorithm became well known when it was rediscovered in 1957 by Robert Prim. Because of this, it is known as **Prim's algorithm** (and sometimes as the **Prim-Jarník algorithm**).

Steps to find minimum spanning tree by prim's algorithm:

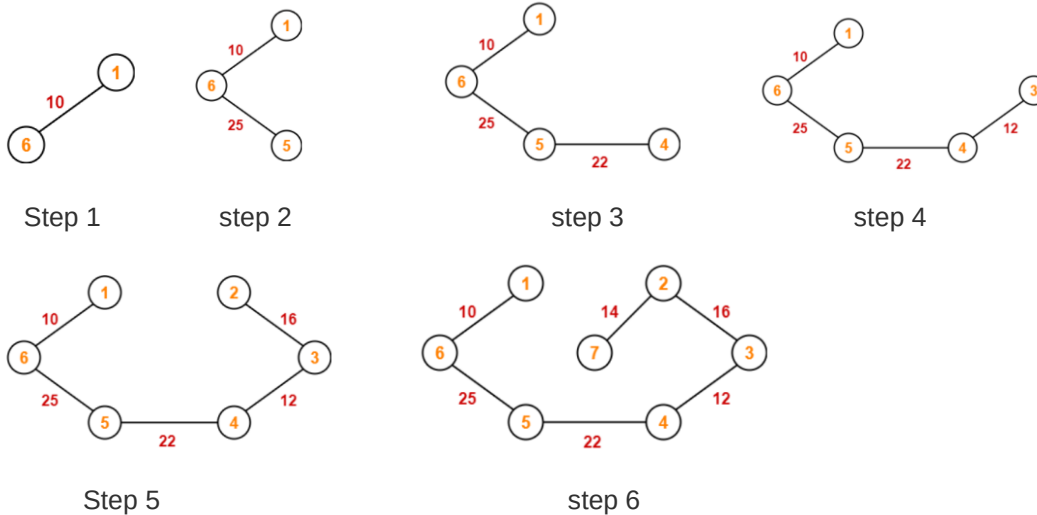
1. Randomly choose any vertex. The vertex connecting to the edge having least weight is usually selected
2. Find all the edges that connect the tree to new vertices.
3. Find the least weight edge among those edges and include it in the existing tree.
4. If including that edge creates a cycle, then reject that edge and look for the next least weight edge.
5. Keep repeating step-3 until all the vertices are included and Minimum Spanning Tree (MST) is obtained.

Example: Find minimum spanning tree and its weight of below graph by using prim's algorithm



Solution:

Let choose vertex 1. Choose the shortest edge from this vertex and add it. And continue with the steps as given above:

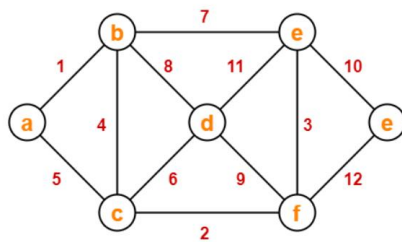


Since all the vertices have been included in the MST, so we stop.
Now, weight of Minimum Spanning Tree = Sum of all edge weights

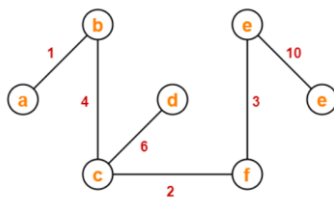
$$= 10 + 25 + 22 + 12 + 16 + 14$$

$$= 99 \text{ units}$$

Example: Find the minimum spanning tree and its weight of below graph.



Solution: By using the above steps we found Minimum spanning tree as shown below



Now, weight of Minimum Spanning Tree= Sum of all edge weights

$$= 1 + 4 + 2 + 6 + 3 + 10$$

$$= 26 \text{ units}$$

Kruskal's algorithm:

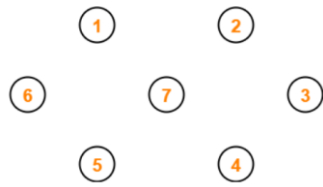
It was discovered by Joseph Kruskal in 1956, although the basic ideas it uses were described much earlier.

Steps to find minimum spanning tree by using Kruskal's algorithm:

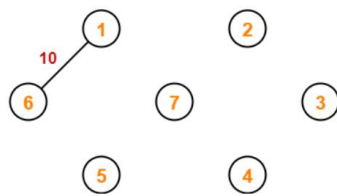
- 1.Sort all the edges from low weight to high weight
- 2.Take the edge with the lowest weight and use it to connect the vertices of graph.
- 3.If adding an edge creates a cycle, then reject that edge and go for the next least weight edge.
- 4.Keep adding edges until all the vertices are connected and a Minimum Spanning Tree (MST) is obtained.

Example: Construct the minimum spanning tree (MST) for the given graph using Kruskal's Algorithm-

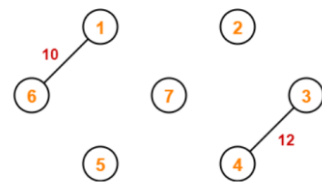
Solution: To construct MST using Kruskal's Algorithm, simply draw all the vertices on the paper. Connect these vertices using edges with minimum weights such that no cycle gets formed.



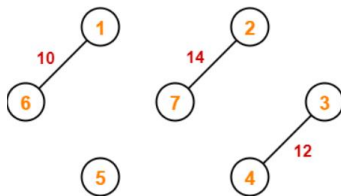
Step 1



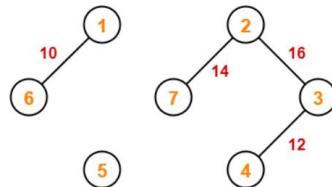
step 2



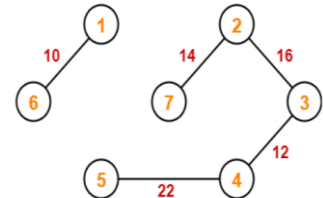
step 3



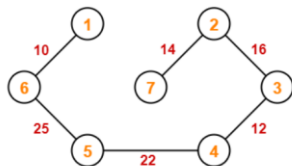
Step 4



step 5



step 6



Step 7

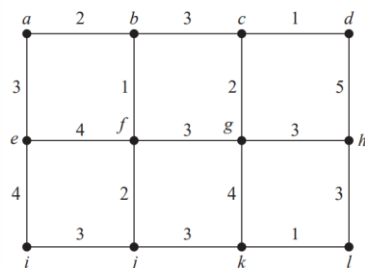
Since all the vertices have been connected / included in the MST, so we stop.

Weight of the MST= Sum of all edge weights

$$= 10 + 25 + 22 + 12 + 16 + 14$$

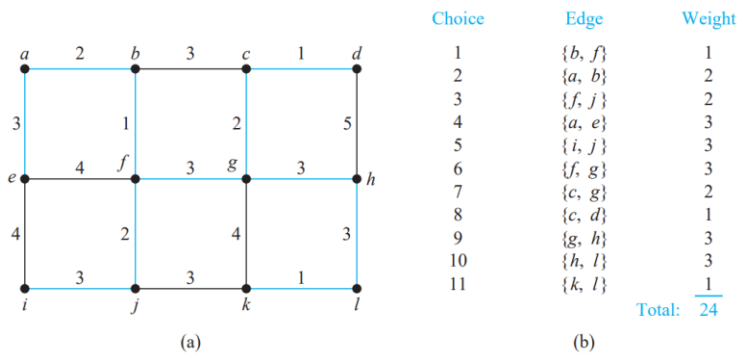
$$= 99 \text{ units}$$

Example: Construct the minimum spanning tree (MST) for the given graph using Prim's algorithm and Kruskal's Algorithm and find the weight of MST

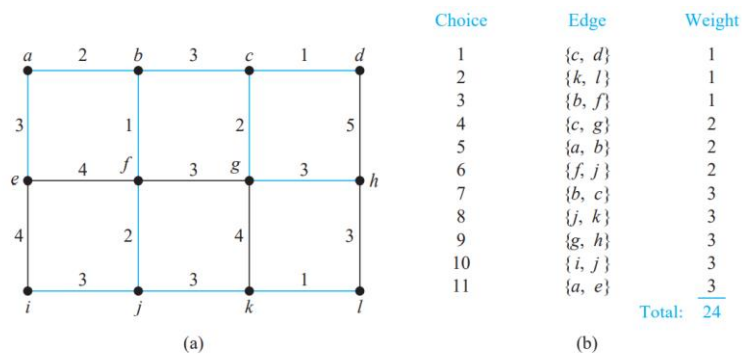


A Weighted Graph.

Solution: MST by using Prim's algorithm:



MST by using kruskal's algorithm:



Note:

Difference between Prim's and Kruskal's algorithms: In Prim's algorithm edges of minimum weight that are incident to a vertex already in the tree, and not forming a circuit, are chosen; whereas in Kruskal's algorithm edges of minimum weight that are not necessarily incident to a vertex already in the tree, and that do not form a circuit, are chosen. Note that as in Prim's algorithm, if the edges are not ordered, there may be more than one choice for the edge to add at a stage of this procedure. Consequently, the edges need to be ordered for the procedure to be deterministic.